

CONTEXTGRAPH: TURNING PAST REPAIRS INTO EXECUTABLE REQUIREMENTS FOR CODING AGENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

A coding agent can fix the reported bug and pass official tests while losing behavior established by an earlier repair. We present ContextGraph, a memory system that connects past repairs to the requirements a new repair should preserve. It stores each requirement with its triggering input, operation, and expected outcome. Retrieved relations help combine historical conditions with the current workflow, producing executable checks whose failures guide further edits. Our system evaluations show gains from experience reuse: ContextGraph resolves 69.2% of SWE-bench Verified tasks versus 61.8% without memory, and reaches 25 resolutions on Related-Lite99 in two passes versus three without memory. Development experiments explain how memory changes repair behavior. Historical requirements recover omitted query strings, uniqueness constraints, and timezone handling; a composed check exposes a failure that separate current and historical examples both miss. Execution feedback also corrects a timezone repair left unchanged by further issue-based review. These findings suggest a useful form of agent memory: retain the conditions behind past fixes so that agents can test what a new repair must continue to do.

1 INTRODUCTION

Coding agents repeatedly encounter related bugs, yet a new issue describes only part of what a complete repair should accomplish. Earlier fixes record argument precedence, constructor conventions, and neighboring behavior that must continue to work. Experience can supply requirements that the next issue leaves implicit.

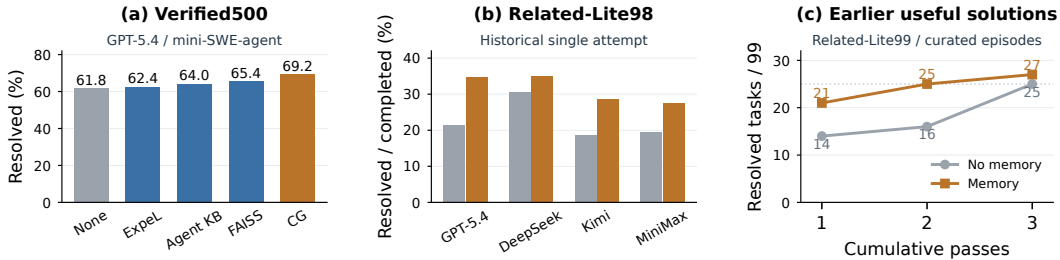


Figure 1: ContextGraph improves repair success across the evaluated settings. (a) SWE-bench Verified500 with GPT-5.4 and mini-SWE-agent; (b) Related-Lite98 with four backbones; (c) cumulative resolutions on Related-Lite99 with DeepSeek V4 Pro. Grey denotes no memory and orange ContextGraph. Each panel uses its own model, memory pool, and attempt setting; Tables 1, 5, and 6 give the corresponding counts.

Consider an administrative redirect that loses a deployment prefix. An agent can restore the prefix yet discard query parameters when appending a slash. An earlier fix of the same API supplies the missing requirement: retain the query string. In our comparison, both repairs pass official verification, but only the repair using history preserves both behaviors. The earlier repair helps by telling the agent what else the code must do.

Reusing repair experience raises three challenges.

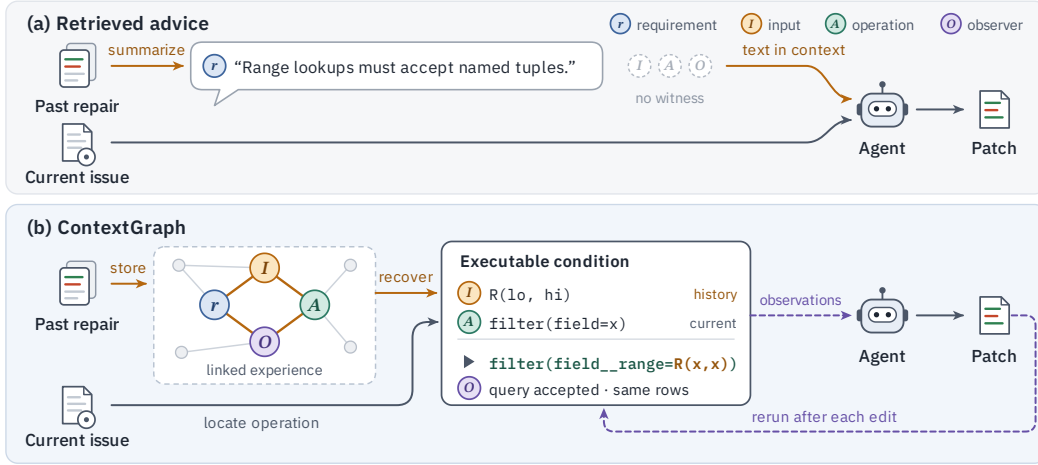


Figure 2: From retrieved advice to an executable requirement. ContextGraph connects requirement r to an input, operation, and observation (I, A, O). It combines the historical input with the current operation: here, $\text{filter}(\text{field}=x)$ becomes $\text{filter}(\text{field_range}=R(x, x))$. The agent uses the observed failure to revise the patch and reruns the check.

Losing the triggering condition. A summary can retain the requirement yet lose the input relation that makes it relevant. An agent may follow advice about constraint deletion but test different columns from those that caused the bug.

Missing an interaction between conditions. A current query and a historical named-tuple example may each pass, while their combination fails. Replaying the examples separately leaves this interaction unchecked.

Leaving a known requirement unimplemented. An agent may understand the requirement yet leave the relevant code unchanged or revise the wrong branch. Repeating the requirement need not resolve the remaining failure.

ContextGraph addresses these challenges through three contributions:

1. **Requirement memory that preserves triggering conditions.** We link each requirement to its executable input, operation, and expected outcome (Section 3.1). The content experiments show how lost column and constructor relations lead to incomplete repairs, even with a relevant requirement (Section 4.3).
2. **Condition composition that checks interactions.** We use these relations to bring a historical trigger into the current operation (Section 3.2). The composition experiments expose a failure missed by separate examples and show how joint checks help recover the omitted behavior (Section 4.4).
3. **Execution feedback that guides requirement-based repair.** We supply the checks and their failures to guide the next edit (Section 3.3). Refinement experiments recover constraint and timezone behavior left unresolved by further textual review (Section 4.5).

System comparisons additionally measure the benefit of experience reuse across repair tasks, backbones, and retry budgets (Figure 1).

2 BACKGROUND AND MOTIVATION

What should an agent remember? Reflexion stores feedback as reflection for later attempts (Shinn et al., 2023). ExpeL distills reusable insights from trajectories (Zhao et al., 2024); ReasoningBank learns reasoning memories from successes and failures (Ouyang et al., 2026); ACE maintains playbooks through generation, reflection, and incremental curation (Zhang et al., 2026).

For software repair, we focus on the behavior learned from a past fix and the conditions under which that behavior matters.

Values alone do not describe a condition. Named tuples and ordinary tuple subclasses can hold identical values but require different constructor calls. Replacing one by the other in an example changes what the example tests. Likewise, a timezone value means something different when it is the database timezone or an explicitly requested destination. Memory must preserve these roles as well as the values.

Two passing examples can hide a failing combination. On a Django repair, a query using a lazy User object passes, and a historical named-tuple range query also passes. Combining the named-tuple range with the lazy User in the current query fails. This observation motivates condition composition: carry the old trigger into the operation being repaired (Figure 2).

3 CONTEXTGRAPH: MEMORY AS EXECUTABLE REQUIREMENTS

3.1 REQUIREMENT MEMORY: PRESERVE TRIGGERING CONDITIONS

To retain what triggers a requirement, we represent a historical repair with original and repaired versions V^- , V^+ as

$$m = (r, W, E^-, E^+), \quad W = (I, A, O), \quad (1)$$

where r states the requirement, I constructs its triggering input, A performs the operation, and O observes the expected outcome. We call W a witness: an executable example that fails before the historical fix and passes afterward. E^- and E^+ record these executions.

A source writer reads the historical issue, patch, and relevant code to propose the requirement and witness. Execution and source review check that the example exercises the intended behavior. The graph links the repair to its requirement, program, executions, and input roles. For a range query, these roles identify the constructor and its two endpoints; for a timezone conversion, they distinguish the stored instant, active timezone, and destination. This gives the next agent both an explanation and a concrete way to test it.

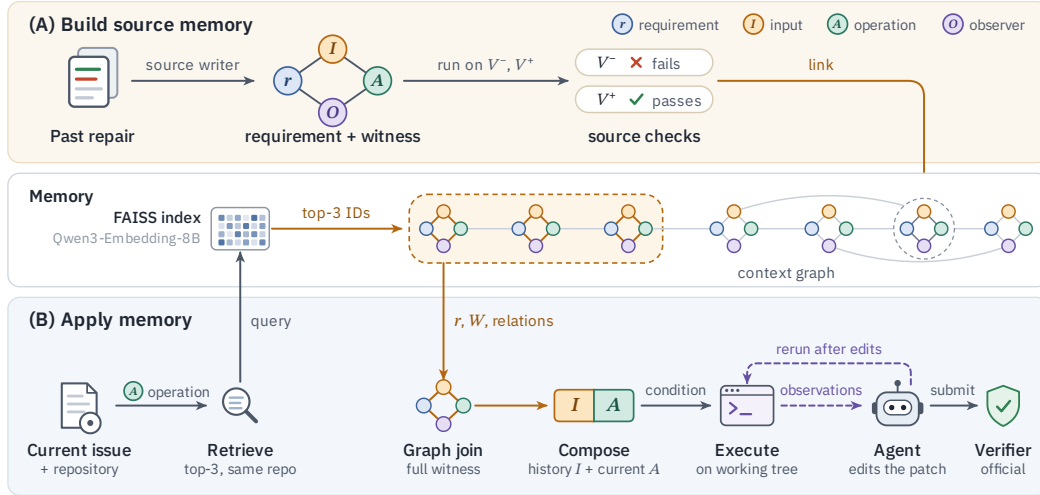


Figure 3: ContextGraph connects source experience to a new repair. (A) A source writer builds a requirement and witness, checked before and after the historical fix. (B) Retrieval selects relevant sources; graph relations support composition with the current operation. The agent uses execution feedback to edit the code, then submits the patch for verification.

Retrieve the requirement together with its context. The implemented retrieval path uses Qwen3-Embedding-8B and FAISS to select three distinct witnessed sources from the same repository (Zhang et al., 2025). Graph joins return their requirements, complete programs, observations,

and input relations. Similarity proposes candidates; the relations explain how to use them. We keep candidates even when their operations differ from the initially localized function: an adjacent operation may carry the requirement the current repair needs to preserve.

Retrieval can also use the agent’s evolving context. The full-context search design lets a code agent inspect complete experiences, follow graph relations, and search again after a failed attempt. It keeps the same requirement and execution interface (Appendix D).

3.2 CONDITION COMPOSITION: CHECK INTERACTIONS

Composition takes a current executable program P_q , a relevant memory m , and the relations that map their inputs to operation arguments. An adapter implements the combination for a supported operation. It retains the current workflow, adds the historical trigger, and returns a check with an expected outcome. A useful check must reach the operation and observe its behavior; an import or fixture error instead calls for fixing the check’s setup.

For a scalar equality lookup, our range adapter uses the historical two-endpoint named tuple R :

$$P_q(x) \mapsto P_q^{\text{range}}(R(x, x)). \quad (2)$$

For a normalized non-null value with equivalent equality and point-range semantics, the expected selection stays the same. The adapter changes the lookup argument while preserving how x is constructed and used. Thus a lazy User inside an annotated subquery remains that object in that query; replacing it with a lazy integer would lose the interaction. An ordinary-tuple range supplies a neighboring check. Our second adapter varies database, ambient, and destination timezones to exercise their different roles.

To select applicable requirements, we compare the historical behavior with the current issue, documentation, tests, and reachable history. Compatible requirements become checks; explicitly changed or unclear behavior is left unenforced. An applicable requirement without an adapter is supplied as cited text. Appendix F gives the decision procedure.

Preparing the current program. In the development cases, P_q combines public issue snippets with researcher-written setup and observations; both adapters are also researcher-written. The broader evaluation specifies a shared agent step to construct a reproducer, prepare its environment, and locate the operation. All methods receive the same prepared program and count its cost in their budget. **Automatic preparation, applicability assessment, and their shared validation remain to be implemented and evaluated.** If preparation fails, repair continues with ordinary tools and applicable text memory.

3.3 EXECUTION FEEDBACK: GUIDE REQUIREMENT-BASED REPAIR

To turn a retrieved requirement into an edit, the coding agent receives the current issue and selected memory, browses the repository, and runs the checks. When a supplied check fails, its program and observations show which behavior the patch still misses. The agent can revise the implementation and rerun the same check. Keeping the input and expected outcome fixed makes feedback comparable across edits. Final patches receive official verification and additional behavioral checks. Those additional evaluation checks are separate from the checks supplied as repair guidance.

4 EXPERIMENTS

We first measure overall repair success. The next three studies test the three contributions in order: what requirement memory must preserve (Section 4.3), what condition composition reveals (Section 4.4), and how execution feedback changes the repair (Section 4.5). We then examine backbones, retries, and memory-pool size.

4.1 SETUP AND EVALUATION

The system comparisons use SWE-bench Verified and SWEContextBench (Jimenez et al., 2024; Zhu et al., 2026). They evaluate summary or episodic memory; the development experiments add

Table 1: SWE-bench Verified500 results with GPT-5.4 and mini-SWE-agent. Rates use 500 tasks; gains are points over no memory.

Method	Resolved	Rate	Gain (pp)
No memory	309	61.8%	—
ExpeL	312	62.4%	+0.6
Agent KB	320	64.0%	+2.2
FAISS	327	65.4%	+3.6
ContextGraph	346	69.2%	+7.4

Table 2: Joint current/historical behavior in complete-agent comparisons. Each recorded repair passes official verification. The columns compare repairs with and without memory; the rows retain their distinct task requirements. [Green additions: Appendix L.](#)

Target	Historical addition	No memory	Memory
Django14404	Preserve query string	Fail	Pass
Django12708	Delete redundant uniqueness	Fail	Pass
Django11138	Honor destination timezone	Fail	Pass
Django12663	Preserve named-tuple range	TBD	TBD

executable witnesses and condition composition. Tables state each comparison’s backbone, source pool, and attempt setting. The development cases use DeepSeek V4 Pro at temperature zero, with matched budgets within each comparison. Detailed task inputs appear in Appendix E.

Official success comes from each benchmark’s verifier. For behavioral analysis, *Joint* checks the current and historical requirements together, *Preserve* checks neighboring behavior that already worked, and *Combined* requires official success and both groups on the same patch. Case tables report the actual checks and attempts; benchmark tables retain their completed and planned task counts. VibeMemBench likewise evaluates coding-agent memory through executable repository outcomes (Fan et al., 2026). Cost measurements include memory construction, retrieval, check execution, and unsuccessful repair attempts.

The additional evaluation compares seven methods on Verified150, Related-Lite99, LoLBench100, and DeepSWE113, and measures behavior on an independently annotated held-out sample. These results remain pending in Appendix G; the red tables collect the work to complete.

4.2 REPAIR EFFECTIVENESS

With GPT-5.4 and mini-SWE-agent, ContextGraph resolves 346/500 tasks versus 309/500 without memory, a gain of 7.4 percentage points (Table 1). It resolves 19 more tasks than FAISS and 26 more than Agent KB (Tang et al., 2025). **Finding:** experience reuse improves repair success in this comparison. The next three sections examine what makes that experience useful.

4.3 REQUIREMENT MEMORY: WHICH CONDITIONS MUST IT PRESERVE?

We test the information needed to use requirement memory (Section 3.1). Three paired agent runs first show what history adds: memory preserves a query string, removes redundant uniqueness while retaining the primary key, and honors an explicit destination timezone (Table 2). Both repairs in every pair pass official verification. These selected development cases use one repair per arm; Appendix E gives the sources and inputs.

To examine the content needed to use a requirement, we supply no memory, a requirement, an implementation reference, or both for the constraint and timezone tasks. All eight repairs pass official verification (Table 3). The constraint Requirement agent tests different columns and misses the same-column trigger; the implementation reference retains that detail. The timezone Requirement agent finds the appropriate helper in the current repository, while the implementation reference leads to errors on named zones.

Table 3: Memory-content interventions: one fresh attempt per input and development task. All eight pass official verification. MySQL and named-zone checks were constructed after inspecting the timezone repairs and applied to all four submissions.

Memory input	Constraint joint	Timezone joint	MySQL cases	Named-zone cases
None	Fail	Fail	Pass	Fail
Requirement	Fail	Pass	Pass	Pass
Implementation	Pass	Pass	Pass	Fail
Both	Pass	Pass	Fail	Pass

Table 4: Added conditions in the original-base Django12663 pair. Both inputs contain the same history, current program and failure, tool, and execution instruction. Only the second adds the composed programs and their failures. Each row counts the named checks, not independent tasks. **Green additions: Appendix L.**

Measured behavior	Current feedback	+ Composed conditions	+ Separate history
Official target verification	Pass	Pass	TBD
Historical named-tuple examples	0/2	2/2	TBD
Withheld nonempty joint queries	2/3	3/3	TBD
Withheld ordinary subclasses	2/2	2/2	TBD
Mechanical scalar / named tuple / tuple	2/3	3/3	TBD

The input’s role also matters. On Django12663, both retrieved packets contain the relevant named-tuple source, yet both initial repairs miss its behavior. A revision using the historical constructor distinction preserves named tuples and ordinary subclasses; a revision expanding all non-builtin sequences into positional arguments breaks the latter (Appendix Figure 4). Requiring an exact operation match also discards this useful source despite its first-place FAISS rank; relations guide adaptation after source selection. **Finding:** a relevant requirement can still lead to an incomplete repair when its triggering relation is lost. Shared columns and constructor arguments are decisive in these cases; more implementation text does not consistently produce a more complete repair.

4.4 CONDITION COMPOSITION: WHAT INTERACTIONS DOES IT REVEAL?

We test what condition composition reveals beyond separate examples (Section 3.2). A history-informed Django12663 patch passes the current lazy-User query and the historical named-tuple example separately, but fails their combination. The range adapter reconstructs this joint check while retaining the lazy User and nested query.

Table 4 compares an original-base pair given the same history, current program, execution tool, and instructions. Adding the composed checks and their failures recovers both historical examples and all three withheld joint queries, while preserving ordinary subclasses.

Timezone composition exposes a similar interaction among database, ambient, and destination roles. Across 72 combinations, Requirement and Both pass all 72 conditions; Implementation fails 32, including 16 passed without memory. The old implementation handles some role combinations but does not capture the full requirement. **Finding:** combining historical triggers with current operations reveals failures that separate examples miss, and gives the agent a check for that interaction. **The planned separate-versus-joint comparison fixes two check slots and the feedback schedule; further contrasts vary relations and composer choice (Table 22).**

4.5 EXECUTION FEEDBACK: HOW DOES IT CHANGE THE REPAIR?

We test whether execution feedback helps implement a known requirement (Section 3.3). Two fresh agents revisit the same failed constraint repair with the same requirement. Requirement-only review fixes the primary-key case but misses the ordinary unique-field case. Adding the source check

Table 5: Related-Lite98, one attempt per task (V_1). Cells show resolved/completed; bold marks the best non-oracle condition by rate. Memory conditions draw from the same 300-summary pool within each backbone.

Memory	GPT-5.4	DeepSeek V4 Pro	Kimi K2.6	MiniMax M2.7
None	21/98	30/98	18/97	19/98
Random summaries	19/97	26/96	25/98	19/97
Mem0	19/98	26/97	25/98	18/98
LangMem	24/98	33/97	24/98	21/98
Oracle summary	35/98	27/97	24/98	15/95
ContextGraph	34/98	34/97	28/98	27/98

Table 6: Related-Lite99 with curated episodic memory. Resolved counts are cumulative. Wins/losses compare ContextGraph with control; p is the paired exact McNemar test. **Green additions: Appendix L.**

Retry budget	No memory	ContextGraph	Wins/losses	p
1 pass	14/99	21/99	7/0	.016
2 passes	16/99	25/99	9/0	.0039
3 passes	25/99	27/99	6/4	.75
4 passes	TBD	TBD	TBD	TBD

and its actual failures leads to name disambiguation that fixes both. On the timezone task, further issue-based review leaves the completed patch unchanged; the historical check and failure lead to a revision that honors the destination timezone.

Feedback also helps use a composed condition. Of four fresh refinements of the Django12663 candidate, history alone passes all seven measured conditions in one of two attempts; the joint program and its failure do so in two of two. All four pass official verification. For the simpler redirect task, a sentence stating the requirement already yields the same correct implementation as requirement-plus-feedback. **Finding:** execution feedback makes a missed condition actionable when further textual review leaves it unresolved; a clear requirement can suffice when the agent readily maps it to the right code.

4.6 MODELS, RETRIES, AND MEMORY-POOL SIZE

Benefits across backbones. In the Related-Lite98 single-attempt comparison, memory methods draw from the same 300-summary pool within each backbone (Table 5), including Mem0 (Chhikara et al., 2025) and LangMem (LangChain, 2025). ContextGraph has the highest resolution rate among the non-oracle conditions on all four models. With GPT-5.4 it resolves 34/98 tasks, compared with 21/98 without memory. The oracle supplies one paired summary; ContextGraph can retrieve several. **Finding:** the benefit of selected experience appears across all four backbones in this setting.

Useful experience reduces the retries needed to reach a solution. A separate Related-Lite99 experiment uses DeepSeek V4 Pro with 500 calls and \$30 per task in both arms. The memory contains 291 curated episodes with reference fixes, verification hints, and pre-base history. Each subsequent pass retries that arm’s unresolved tasks. ContextGraph reaches 25 resolutions in two passes; the no-memory agent needs three (Table 6). The gain is largest in the first two passes and narrows after the third. Excluding ten targets with near-duplicate source fixes retains the first-pass gain, 19/89 versus 13/89 ($p = .031$). **Finding:** memory is most useful early, reaching in two passes the 25 resolutions that the no-memory agent reaches in three.

Pool size and source coverage. With the GPT-5.4 agent and retrieval procedure fixed, the 300-summary pool produces the best observed result, 34/98, while smaller pools yield 21–27 resolutions (Table 7). Pool growth changes both useful source coverage and competition during ranking. The planned distractor and source-removal experiment examines these effects separately (Table 31).

Finding: the largest tested pool performs best, but the intermediate results are not monotonic; pool size alone does not determine memory utility.

Table 7: Related-Lite98 memory-pool ablation, GPT-5.4, single-shot V_1 . The separate best-of-two V_2 result is in Appendix Table 10. **Green additions:** Appendix L.

Number of summaries	50	100	200	300
Resolved/completed	21/96	27/98	26/97	34/98
Resolution rate	21.88%	27.55%	26.80%	34.69%
Source coverage (%)	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
USD/resolution	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

These findings motivate measuring memory quality together with computation. **The cost study counts construction, selection, checks, and every repair attempt, and compares time and cost per solution (Tables 29 and 32).** Additional system results appear in Appendices A and C.

5 DISCUSSION AND RELATED WORK

Learning from repair experience. SWE-Exp distills successful and failed repair trajectories into reusable knowledge (Chen et al., 2025). ExpeRepair combines concrete demonstrations with reflective insights (Mu et al., 2026); MemGovern organizes GitHub history into experience cards that agents search and browse (Wang et al., 2026). STAIR abstracts trajectories into repair plans at multiple levels (Xu et al., 2026). These approaches provide ways to reuse what earlier repairs have taught an agent. MemOp optimizes memory through its effect on downstream tasks, while ExpGraph learns retrieval over linked experiences (Guo et al., 2026; Feng et al., 2026). Recent AdaRepair-Mem also selects experience according to repository coverage, quality, and repair stage (Luo et al., 2026). ContextGraph focuses on carrying a historical requirement and its triggering conditions into the operation being repaired.

Organizing and managing memory. MemGPT manages information across context and external storage (Packer et al., 2023); SWE-MeM learns when and how to compress the history of a long coding task (Gao et al., 2026). A-MEM dynamically links and updates memory notes, while Zep represents facts and their temporal relationships (Xu et al., 2025; Rasmussen et al., 2025). HippoRAG uses graph connections for associative retrieval (Gutiérrez et al., 2024). These designs explain how information can remain available and connected as an agent works. ContextGraph uses input and operation relations to make retrieved repair experience executable.

Turning experience into checks. RETester adapts historical bug reports into tests for refactoring engines (Wang et al., 2025). MIRA separates reusable requirements from SQL corrections, and TRACE compiles user corrections into runtime checks (Liu et al., 2026; Zhou et al., 2026). Our development cases motivate a further step: combine the historical trigger with the current workflow, since each example can pass separately while their combination fails. Retaining the trigger and its expected outcome gives the agent a concrete repair objective. The current adapters cover two Django operation families; the additional evaluation examines broader coverage and independently checked behavior.

6 CONCLUSION

ContextGraph turns past repairs into requirements that a coding agent can apply and test. It preserves triggering inputs with expected behavior, composes those conditions with the current operation, and uses execution failures to guide further edits. System comparisons show higher repair success and earlier resolution under retries. Development experiments show what these design choices contribute: retaining a decisive trigger, exposing an interaction missed by separate examples, and turning a remaining violation into a concrete next edit. Together, these ideas let an agent use past repairs as concrete requirements for its current task.

REFERENCES

- Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution, 2025. URL <https://arxiv.org/abs/2507.23361v2>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- Liyang Fan, Yingcheng Shi, Yongbin Li, Chenghao Sun, Xin Chen, Xander Xu, Hu Wei, Shiwen Ni, Min Yang, and Jieping Ye. VibeMemBench: Evaluating memory systems for coding agents on real repository coding tasks, 2026. URL <https://arxiv.org/abs/2609.23570v1>.
- Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, Zhigang Hua, Yan Xie, Shuang Yang, Ge Liu, and Jiaxuan You. ExpGraph: Model-agnostic experience learning with graph-structured memory for LLM agents, 2026. URL <https://arxiv.org/abs/2605.30712v1>.
- Shuzheng Gao, Wenhao Zeng, Zhaojian Yu, Jianqiao Wangni, Chaozheng Wang, Kai Cai, Shilin He, and Michael R. Lyu. SWE-MeM: Learning adaptive memory management for long-horizon coding agents, 2026. URL <https://arxiv.org/abs/2606.28434v1>. Version 1.
- Xuehang Guo, Zora Zhiruo Wang, Qingyun Wang, Graham Neubig, and Xingyao Wang. Enhancing software engineering through closed-loop memory optimization, 2026. URL <https://arxiv.org/abs/2606.05646v1>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1-Abstract.html.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- LangChain. LangMem, 2025. URL <https://github.com/langchain-ai/langmem>. Software library.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- Z. C. Luo, J. C. Guo, W. J. He, S. Y. Wang, J. C. Yu, F. M. Zhao, Y. Chen, T. Cao, L. Q. Liu, N. Zheng, W. Xu, J. Jiang, and Z. M. Zhao. AdaRepair-Mem: Adaptive experience orchestration for repository-level program repair, 2026. URL <https://arxiv.org/abs/2609.20130v1>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=jL7fwchScm>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.

- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory, 2025. URL <https://arxiv.org/abs/2501.13956>.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. Agent KB: Leveraging cross-domain experience for agentic problem solving, 2025. URL <https://arxiv.org/abs/2507.06229>.
- Haibo Wang, Zhuolin Xu, and Shin Hwei Tan. Testing refactoring engine via historical bug report driven llm, 2025. URL <https://arxiv.org/abs/2501.09879v2>.
- Qihao Wang, Ziming Cheng, Shuo Zhang, Fan Liu, Rui Xu, et al. MemGovern: Enhancing code agents through learning from governed human experiences, 2026. URL <https://arxiv.org/abs/2601.06789v2>.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/19909c36f51abc4856b4560aff3d36d6-Abstract-Conference.html.
- Yisen Xu, Jiayuan Zhou, Ruiqi Pan, and Tse-Hsun Chen. Reusing past repairs through hierarchical trajectory abstraction for coding agents, 2026. URL <https://arxiv.org/abs/2607.29658v1>. Version 1.
- Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, et al. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2510.04618v3>.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176v3>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 2024. doi: 10.1609/aaai.v38i17.29936. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29936>.
- Yujun Zhou, Kehan Guo, Haomin Zhuang, Xiangqi Wang, Yue Huang, Zhenwen Liang, Pin-Yu Chen, Tian Gao, Nuno Moniz, Nitesh V. Chawla, and Xiangliang Zhang. Getting better at working with you: Compiling user corrections into runtime enforcement for coding agents, 2026. URL <https://arxiv.org/abs/2606.13174v1>.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL <https://arxiv.org/abs/2602.08316v3>. Version 3.

A ADDITIONAL SYSTEM COMPARISONS

These tables give additional system comparisons and their experiment settings. The earlier Verified500 run used 75 steps per task; the main text reports the later 346/500 comparison. Related-Lite results cover summary memory, episodic memory, and memory-interface comparisons.

Table 8: Historical SWE-bench Verified500, GPT-5.4, 75 steps and uncapped per-task cost. Rates use all 500 tasks, including unfinished attempts. Wins/losses and exact McNemar p compare each method with no memory.

Method	Resolved	Rate	Gain (pp)	Wins/losses	p
No memory	309	61.80%	—	—	—
ExpeL	312	62.40%	+0.6	43/40	.826
FAISS	327	65.40%	+3.6	49/31	.057
ContextGraph	318	63.60%	+1.8	50/41	.402

Memory configurations. The Verified500 system indexes trajectory fragments, rules, and communities with dense, lexical, and graph retrieval. The Related-Lite98 system indexes 300 Experience summaries with repository-filtered lookup and a memory tool; its production repository-filtered route differs from generic channel fusion. The Related-Lite99 episodic system serves reference fixes, verification hints, and pre-base history from 291 episodes at pre-plan and post-error query points. These configurations precede the Qwen executable-witness graph.

Task counts and attempts. Verified500 uses a fixed 500-task denominator; completed counts were 486 for no memory, 488 for ExpeL, 490 for FAISS, and 484 for ContextGraph. Related-Lite98 excludes one shared image-build failure from the original 99-task manifest and reports resolved/completed in each arm. Its V_1 regime uses one attempt; the historical V_2 aggregate includes failure reruns. The main-text pool-size table uses V_1 only: the old delta column used the V_2 control rate and is omitted. The MiniMax M2.7 campaign used a gateway-compatibility shim.

Related-Lite99 source content. The episodic campaign served static source episodes: its original online deposit path omitted fields needed for retrieval. The historical overlap analysis identified ten targets with near-duplicate source fixes (token-Jaccard at least 0.85); removing them gives an 89-task frame. DeepSeek V4 Pro retains a first-pass gain of 19/89 versus 13/89 ($p = .031$). The full-set cumulative two-pass gain of nine exceeds the campaign’s preregistered threshold of eight.

Table 9: Historical curated-episode Related-Lite99 replication, one pass per backbone. Last column gives recorded control/memory counts and paired p after removing ten near-duplicate targets; arm-specific completed denominators for that subset are not provided in the original table.

Backbone	Control	Memory	C/M only	p	Reduced set
DeepSeek V4 Pro	14/99	21/99	0/7	.016	13/19 ($p = .031$)
DeepSeek V3.2 exp	16/98	20/99	4/8	.39	15/18 ($p = .55$)
GLM 5.2	17/99	24/99	1/8	.039	15/21 ($p = .070$)
Kimi K3	29/99	31/99	3/5	.73	26/28 ($p = .73$)

Table 10: Historical Related-Lite98 GPT-5.4 aggregate with failure reruns (V_2 reporting regime). The historical table alone does not supply uniform per-arm retry accounting; Table 5 gives the separate single-shot V_1 comparison. **Green additions: Appendix L.**

Memory	Resolved/completed	Rate	Attempts/task
Random summaries	19/97	19.59%	<i>TBD</i>
Mem0	19/98	19.39%	<i>TBD</i>
LangMem	24/98	24.49%	<i>TBD</i>
No memory	25/98	25.51%	<i>TBD</i>
Oracle summary	35/98	35.71%	<i>TBD</i>
ContextGraph	41/98	41.84%	<i>TBD</i>

Table 11: Historical Verified development subset. Rates use completed evaluations. The preprint transcribed this table from an experiment screenshot; 66.6% retains the recorded display precision.

Method	Completed/planned	Resolved	Recorded rate
ContextGraph	48/50	32	66.6%
ExpeL	50/50	32	64.0%
FAISS	49/50	30	61.2%
No memory	50/50	29	58.0%

Table 12: Historical SWE-bench-Pro50, recovered from a disabled preprint section. Memory arms share a 3,183-strategy pool and a \$10 per-task cap; no memory is uncapped. Rates use completed evaluations; costs are recorded totals under these different budgets. **Green additions: Appendix L.**

Method	Completed/planned	Resolved	Rate	Cost (\$)
ContextGraph	47/50	11	23.40%	41
FAISS	47/50	5	10.64%	68
No memory	46/50	2	4.35%	1,686
<i>Random strategies</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 13: Historical trajectory-graph snapshot, using text-embedding-3-large vectors with 3,072 dimensions. The new executable-witness graph uses a separate Qwen embedding space. **Green additions: Appendix L.**

Node label	Count	Role	With vector
Trajectory	1,795	Source episode	<i>TBD</i>
Fragment	13,813	Action and observation slice	<i>TBD</i>
ErrorPattern	586	Error type and keywords	<i>TBD</i>
Strategy	9,600	Extracted rule	<i>TBD</i>
CanonicalRule	7,835	Consolidated rule	<i>TBD</i>
PlaybookEntry	8,910	Retrieval-ready entry	<i>TBD</i>
Community	781	Linked-fragment summary	<i>TBD</i>
ProblemSummary	1,795	Episode problem abstraction	<i>TBD</i>
Total nodes	45,115		<i>TBD</i>
Total edges	73,975	Seven typed relations	n/a

B EARLIER INTERFACES AND DEVELOPMENT COMPARISONS

Table 14 compares six interfaces in a Related-Lite99 experiment: a separate DeepSeek V4 Pro campaign with 500 calls and the last five observations retained. Its frozen verdict requires fail-to-pass success and does not veto every pass-to-pass regression. The 88-task frame belongs to this campaign’s source-overlap analysis.

Table 14: Earlier Related-Lite99 memory interfaces and their original 88-task sensitivity frame. Gains and paired exact tests use the full 99-task frame.

Memory	Resolved/99	Resolved/88	Gain (pp)	<i>p</i>
No memory	17	16	—	—
Flat patches	18	18	+1.01	1.000
Patches + procedure card	18	15	+1.01	1.000
Graph	12	12	-5.05	.227
Rules	13	12	-4.04	.424
File list	9	8	-8.08	.057

Table 15: Django14404 development comparison. All candidates pass the official 2 fail-to-pass and 323 pass-to-pass tests.

Candidate	Prefix	Query	Prefix + query
Initial repair	Pass	Fail	Fail
Issue-based revision	Pass	Fail	Fail
Historical patch revision	Pass	Pass	Pass
Executed relation revision	Pass	Pass	Pass

Intervention scope. Content comparisons start fresh agents from the original development target; refinement comparisons start from a common completed candidate. Additional MySQL, named-zone, and ordinary-subclass checks were constructed after inspecting the corresponding repairs, then applied across their comparison group. The composition pair holds history, current example and failure, and the execution interface fixed; the added input is the composed condition.

Operation-dependent preservation. In Matplotlib20826, clearing and replotting twin axes restores finite data bounds, so failure of an isolated historical plotting example does not describe the state after that workflow. In Django16819, canceling a same-name index remove/add pair loses a required change of indexed column. These observations motivate applying memory conditions along the current operation sequence.

Four-benchmark extension. The new matched no-memory, FAISS, and Graph evaluation targets Verified150, Related-Lite99, LoLBench/CAPBench100, and Datacurve DeepSWE113. These four new-method result sets are incomplete. The closed DeepSWE Kea development baseline passes 9/12 fail-to-pass and 139/139 pass-to-pass tests (binary reward 0). The closed JDK5 LoLBench baseline originally submitted no artifact; a separate no-inference recovery of its recorded patch passes 0/5 fail-to-pass and 51/51 pass-to-pass tests (binary reward 0). These are development outcomes. The historical Verified500 table is a different evaluation from Verified150.

C ADDITIONAL NEW EXPERIMENT RESULTS

These experiments extend the Verified500 comparison in Table 1 across agent backbones and measure both task resolution and token use. Earlier campaigns appear in Appendix A.

Table 16: New SWE-Context comparison. Entries reproduce the reported resolved/total counts. **The run configuration and benchmark manifest are TBD.** Green additions: Appendix L.

Method	Resolved/total	Tokens/task	USD/resolution
No memory	21/98	<i>TBD</i>	<i>TBD</i>
Free summaries	19/98	<i>TBD</i>	<i>TBD</i>
Mem0	19/98	<i>TBD</i>	<i>TBD</i>
LangMem	24/98	<i>TBD</i>	<i>TBD</i>
ContextGraph	35/98	<i>TBD</i>	<i>TBD</i>

Table 17: New Gemini 3.1 Pro/live-SWE-agent comparison. **Benchmark identity and sample count are TBD.** The gain column reports relative improvements from the supplied rates. Green additions: Appendix L.

Metric	No memory	ContextGraph	Relative gain
pass@1	82.3%	88.9%	+8.0%
pass@3	82.6%	90.4%	+9.4%
pass@5	82.8%	91.7%	+10.7%
Tokens/task	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Token efficiency. The new summary reports 236K tokens per task without memory and 201K with ContextGraph, a 14.8% reduction (approximately 15%). **The benchmark, sample count, and token accounting scope remain TBD.**

Table 18: New Codex/GPT-5.5 comparison, preserving the supplied counts and denominators. **Benchmark identity and the pass@3 sampling unit are TBD.** Green additions: Appendix L.

Reported metric	Codex + GPT-5.5	With ContextGraph	FAISS Flat
pass@1	1/20	3/20	<i>TBD</i>
pass@3	1/20	5/30	<i>TBD</i>
Tokens/task	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
USD/resolution	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

D FULL-CONTEXT CODE-AGENT RETRIEVAL

This extension uses the same experience records, applicability checks, composer, and budget as fixed-candidate retrieval. The retrieval problem is to find experience that advances the current repair. A short issue embedding can omit the information that makes a memory useful: the code already inspected, a failed approach, an observed exception, or a condition that a partial repair has broken. We therefore propose giving the selector the coding agent’s complete current-task context and tools for exploring the experience pool.

Let C_t contain the task and conversation, inspected code, edits, actions, and observations available at step t . The selector operates on C_t and a frozen experience pool $\mathcal{M}$. It can search with FAISS or lexical queries, open complete source records, follow operation and input relations, and compare the requirements supplied by different experiences. Its output is a set of source IDs, their expected contribution to the repair, and the records or executable conditions supporting that choice. Selection can recur when new execution feedback changes what the repair needs.

Full context describes the selector’s access to the current task; it does not require placing every experience into one prompt. Search and record reading make the whole admitted pool accessible. The returned material retains the source identity and can use the same condition-composition and execution interface as fixed-candidate retrieval.

Current implementation. The repository already provides context-aware candidate reranking and a selector over all 300 compact experience cards. These prototypes cap task or working-state text at 4,000 characters and limit candidate content. They provide starting points for the proposed full-context, iterative code-agent search; the implemented source-witness path currently uses the FAISS candidate policy described in Section 3.

Table 19: Does full-context search find more useful experience? Use 24 development tasks and three repetitions with the same starting solver context, model, bank, output format, maximum three sources, composer, and total selection-plus-repair budget. B, D, and E share the search controller. B/D can read the same ten FAISS candidates; E can search the full pool. D–B tests task context and E–D tests pool access. C uses the existing all-card selector. Utility and Combined follow the shared development suite; search cost includes every selector call.

Selection policy	Utility@3	Combined	Search cost (\$)
A. FAISS top-three	TBD	TBD	TBD
B. Task-only agent; fixed candidates	TBD	TBD	TBD
C. Task-only selector; all experience cards	TBD	TBD	TBD
D. Full-context agent; fixed candidates	TBD	TBD	TBD
E. Full-context agent; iterative pool search	TBD	TBD	TBD

E DEVELOPMENT CASES AND INPUT CONSTRUCTIONS

This appendix gives the inputs, source requirements, and repair observations for the three design studies in Sections 4.3–4.5.

E.1 REQUIREMENT MEMORY: PRESERVING TRIGGERING CONDITIONS

The executable-memory experiments use a Verified split of 280 source tasks, 70 development tasks, and 150 held-out evaluation tasks. The cases below are development interventions with DeepSeek V4 Pro through OpenRouter, temperature zero and matched budgets within each comparison. They measure whether history adds a required behavior to the submitted repair. Table 2 presents three such additions. Django14404 uses the first-ranked source issue and patch; Django12708 and Django11138 come from a fixed six-task cohort receiving three unchanged Flat rules per task. Both arms pass official verification on all six cohort tasks. The historical requirements distinguish the completeness of those repairs.

An omitted behavior of the same API. Django14404 reports that administrative redirects lose a deployment prefix. Both agents fix the prefix and pass official verification, but only the repair informed by source16612 also preserves the query string. A joint input combines the prefix with repeated and percent-encoded query values, `keep=a%2Fb&keep=c%20d`. Only the history-informed repair preserves both.

A neighboring operation. Django12708 reports ambiguous deletion of an ordinary index when a unique constraint uses the same columns. The first retrieved source, 15732, instead concerns deletion of a unique constraint sharing its column with a primary key. Both repairs fix ordinary-index selection; only the memory-informed repair also excludes primary keys when deleting unique constraints. The source-generated PostgreSQL check runs unchanged on both target repairs. A separate combined input creates an ordinary index, redundant uniqueness, and a primary key on the same column. Both repairs preserve ordinary-index deletion, but only the memory-informed repair also deletes redundant uniqueness while retaining the primary key and data. History supplies a requirement for the neighboring operation.

An independent argument of the same operation. Django11138 requires date conversions to respect the database’s source timezone. Source13363 requires `TruncDate` and `TruncTime` to honor an expression’s explicit destination timezone. Both agents repair the source timezone, but only the memory-informed repair also honors the destination. A source-generated check distinguishes the historical versions and runs unchanged on the target repairs. On an independent SQLite input combining a UTC+1 database with an explicit New York destination, only the memory-informed repair returns the correct date and time. The issue and history constrain different ends of the conversion.

A requirement can suffice without the source implementation. On the redirect task, Flat memory and its original issue/patch replacement produce identical repairs preserving the joint behavior. Two new agents then revise the ordinary patch with a source-generated requirement: the redirect must include the original query string. One also receives the check and its failed observations. Both submit identical repairs passing official and joint checks. The sentence-only agent finds `get_full_path()` in the current repository: memory supplies the behavior, and the agent locates its implementation.

Which information makes the requirement usable? We select the constraint and timezone benefit cases and run four fresh complete agents per task, fixing the single historical source and its delivery before editing. Two source-only constructions produce a behavioral requirement and an implementation reference; the combined input concatenates them unchanged. All eight repairs pass official verification (Table 3). On Django12708, only Implementation and Both also remove redundant uniqueness while preserving the primary key. The Requirement agent tests a field with `unique=True`, but places `unique_together` on a different, two-column set. This nearby passing test loses the same-column condition stated in memory. On Django11138, all three memory inputs preserve the explicit destination timezone, while the fresh no-memory repair omits it. The

	Official verification	Historical named tuples (2)	Named tuple + lazy User (1)	ADDITIONAL DIAGNOSTIC Ordinary tuple/list subclasses (2)
Memory supplied for revision				
Original issue + patch	Pass	Pass	Pass	Pass
Requirement sentence	Pass	Fail	Fail	Pass
Requirement + check + execution failures	Pass	Pass	Pass	Fail

Figure 4: Memory content changes which behaviors a repair preserves. One revision per input starts from the same Django12663 candidate and source13590. Parentheses give case counts; each cell reports all cases in its group. The ordinary-subclass inputs were chosen after inspecting the feedback repair. All three repairs pass official verification, while only original history passes every displayed behavior.

Requirement agent finds `self.get_tzname()` in the target implementation without being given that helper in memory.

The timezone repairs also differ on further conditions. Both still generates an unnecessary MySQL conversion between identical zones; Implementation instead misinterprets a named Paris source timezone, producing New York time 19:21 instead of 17:30 on a summer input. Requirement passes both additional diagnostics. These concrete omissions distinguish carrying a requirement, applying its triggering condition, and implementing the resulting behavior correctly. Two subsequent original-base agents receive the unchanged constraint Requirement, with or without a source-validated example; both pass official and joint checks. The same requirement succeeds when its trigger is reconstructed correctly; preserving that input construction is a useful role for the graph.

Preserve the input relation. On Django12663, both a three-source code-connected packet and a ten-source similarity packet contain the historical named-tuple repair, source13590, yet neither initial repair recovers its behavior. Three fresh revisions start from the same candidate and source (Figure 4). Original history and execution feedback recover named tuples; the requirement sentence alone does not. The feedback repair expands every non-builtin sequence into positional arguments, which also breaks ordinary tuple and list subclasses. Original history retains the constructor distinction and passes all displayed behaviors. This is precisely the relation the graph should carry: the argument convention of the input’s type, alongside its endpoint values.

Retain candidates that can supply an omitted condition. In this same case, demanding an operation match from the initial public-code localization excludes source13590, which FAISS ranks first. Keeping the similarity candidate lets the graph return its input relation and construct the joint program described next. This motivates the division in Figure 3: FAISS proposes sources; graph relations support their adaptation to the current operation. The usefulness of a candidate is established by what its condition adds to the repair.

E.2 CONDITION COMPOSITION: CHECKING INTERACTIONS

Join the historical input to the current operation. The current issue involves a lazy User in a nested query; history contributes a named-tuple range. On a separate history-informed repair, the current scalar query and the historical range query each pass, while a query combining them fails. The composed condition reveals an omission in the second container reconstruction.

The deterministic adapter in Equation 2 reaches the same omission while retaining the public program’s lazy User and nested annotation. It produces a named-tuple point range and an ordinary-tuple neighbor. Graph delivery subsequently reconstructs this same program from the retrieved input relation. It operationalizes the condition tested in the repair comparison.

A matched comparison tests the added condition and feedback. A new original-base pair receives identical history, public scalar program, its observed failure, a callable execution tool, and instructions to run the supplied conditions before editing and submission. The augmented arm also receives the two composed programs and their failures. Both repairs pass official verification (Table 4). The current-feedback repair fails both historical examples and the withheld named-tuple/lazy-User query; the augmented repair passes those behaviors and preserves the ordinary subclasses. The augmented agent runs the joint input before editing and again after removing a global conversion that broke an existing test. Its remaining range-endpoint failure directs the final correction.

Compose roles to expose a broader requirement. The timezone relation provides another example. A fixed adapter varies the historical witness’s three zones across database, ambient, and destination roles and shifts the input by 182 days. Of 72 combinations, the source patch changes 20 from fail to pass; 20 pass on both versions and 32 fail on both. Applying the family to the four timezone content interventions shows that Requirement and Both pass all 72. Implementation passes the 40 source-supported combinations but fails the remaining 32, including 16 passed without memory. Role composition exposes how the current source-timezone requirement interacts with the historical destination requirement. A complete repair can therefore satisfy combinations beyond those corrected by the historical implementation.

E.3 EXECUTION FEEDBACK: GUIDING REQUIREMENT-BASED REPAIR

The addition can guide further repair. Two fresh agents start from the same completed ordinary timezone patch. Issue-based additional review returns it unchanged; supplying a generated historical check and its failed observations leads the other agent to use `self.get_tzname()` in both truncation methods. Both revisions pass official verification, but only the latter passes the historical and joint inputs. The intervention supplies corrective information that further issue-based review does not recover in this comparison.

Execution feedback makes an omitted condition concrete. Two fresh agents refine the same original failed Requirement candidate, both receiving its requirement again. One also receives the unchanged source-generated check and its actual failures for a field with a primary key or `unique=True`. Requirement-only review excludes the primary key but still fails the ordinary unique-field condition. Feedback adds generated-name disambiguation and passes both conditions. Both revisions pass official verification and the withheld joint sequence. The added behavior is recovery of the ordinary unique-field condition identified by feedback.

Feedback from the composed query. The Django12663 example also provides a refinement comparison. Four fresh refinements of its history-informed candidate illustrate the use of feedback: history alone yields one of two repairs passing all seven measured conditions; adding the constructed joint program and its failed observation yields two of two. All four pass official verification. Both augmented agents execute the joint input and repair the second reconstruction.

F IMPLEMENTATION AND EVALUATION DETAILS

F.1 RETRIEVAL AND REPAIR SETTINGS

The implemented witness-retrieval path selects the top three same-repository sources using Qwen3-Embedding-8B and FAISS. It returns complete records and typed input relations. The broader comparison uses the existing 273-source bank plus validated pre-base additions, with the bank and adapters fixed before evaluation. Iterative search changes the selection procedure; sequential updating changes when new experience enters memory. Both use the same requirement representation and composition interface.

The experiment configuration records source and target IDs, model and agent versions, index and adapter versions, prompts, budgets, and verifiers. This makes the retrieval, behavioral, and cost comparisons reproducible.

F.2 PREPARING THE CURRENT EXECUTABLE PROGRAM

The development range example extracts model and test snippets from the public issue and adds researcher-written setup and observation code. The range and timezone adapters are also researcher-written. **The following automatic preparation and applicability workflow is specified for the additional evaluation and still requires implementation and validation.**

Given an issue and the original repository, the coding agent first inspects public code, tests, and documentation. It extracts or constructs a reproducer, prepares dependencies and objects, and identifies the operation to combine. The output contains setup, exercise, and observation functions; the execution command and environment; input roles; and the public evidence for the expected outcome. An initial execution checks that the program reaches the intended operation. A failure of the reported behavior is useful; a setup failure means preparation is incomplete.

This preparation runs before memory is supplied. All methods receive the same program, execution record, or failure status for each task and repetition. Its full resource use counts against every method’s total budget. When the artifact is physically reused, the campaign ledger records its actual charge once while the per-method estimate includes the full preparation cost. Preparation has fixed tool, token, and time limits. It uses public information and leaves the candidate implementation unchanged. If it fails, the agent continues ordinary repair work, with any applicable requirement supplied as text. Researchers do not prepare individual fixtures in this automatic setting.

F.3 DECIDING WHETHER HISTORY STILL APPLIES

The assessor compares the retrieved requirement with the current issue, version-matched documentation and tests, and reachable pre-base history. Acceptance requires compatible input and operation semantics and current evidence for the expected behavior. An explicit behavior change rejects the old requirement; missing or conflicting evidence produces an uncertain decision. Only accepted requirements become executable checks. An accepted requirement without a supported adapter is delivered as cited text.

Each decision records the supporting source and current evidence, the input mapping, and the reason. The automatic assessor uses the same backbone as the solver. Independent annotators evaluate these decisions and the resulting repairs; their labels and checks remain private. Appendix K.2 describes this evaluation.

F.4 COMPOSITION INTERFACE

The composer takes the current program P_q , memory m , typed bindings, and an adapter registry. Bindings describe constructors, input values and types, operation arguments, current objects, and their source locations. It returns a check, its observer, the mapping used, and a construction status. The steps are:

1. Match an adapter to the current operation and its type requirements.

2. Map current objects to historical input roles, preserving both triggers and the current operation sequence.
3. Construct the program and check its fixture, bindings, and observer. The expected outcome comes from the requirement, independently of the candidate patch.
4. Execute the check and return a behavioral failure as repair feedback. Record setup or observation errors separately so they can be distinguished from failures of the requirement.

The range adapter transforms the public query’s syntax tree. It preserves the lazy User and nested query while changing the terminal lookup to a point range. Equality and range must select the same normalized non-null value. The timezone adapter instead constructs combinations of database, ambient, and destination roles. Source extractors recover the bindings these adapters support. Adding an operation family requires another adapter and its development tests.

F.5 COVERAGE AND MECHANISM COMPARISONS

Before any run, classify each target from its public issue and repository as supported by the adapters, outside their scope, or uncertain. Retain the same groups when reporting preparation, delivery, repair success, and cost. Failures and text fallback stay in their assigned groups.

The relation ablation gives the same composer either stored bindings or bindings reconstructed from text. Both use the same record schema, source IDs, applicability decisions, validator, and budget. Withheld relations are also omitted from tool explanations. Another contrast supplies the same bindings to an adapter or a free-form composer. A third supplies execution observations before editing; ordinary execution tools remain available to all arms. The separate-versus-joint comparison holds two check slots and feedback rounds fixed. These comparisons measure the role of each choice on the development tasks (Table 22).

G ADDITIONAL EVALUATION

The tables in this appendix contain experiments still to complete. They extend the existing results with a common four-benchmark comparison, independent behavioral evaluation, and controlled mechanism tests.

G.1 REPAIR SUCCESS AND BEHAVIORAL COMPLETENESS

The seven methods share the coding agent, backbone, historical source pool, target tasks, tools, and total budget. ContextGraph and FAISS Flat retrieve the same three source IDs; ContextGraph additionally uses their relations to compose checks. The comparison uses a fixed memory bank. Updating memory after each task is a separate experiment with the same allowed feedback and task order for every method.

Independent evaluators specify current requirements on up to 40 untouched tasks per benchmark. Their checks are sealed before seeing the new patches and are withheld from the coding agents. Combined success requires official resolution, the current/historical combinations, and preservation of selected neighboring behavior. Official success with a failed valid requirement counts as an omission. Appendix K.1 gives the sampling and annotation procedure.

Table 20: Matched first-pass official resolution on four benchmarks. Cells report resolved/planned and assessed/planned. The last row gives the paired ContextGraph–Flat difference in percentage points, its 95% interval, and the jointly assessed count.

Method	Verified	Related-Lite	LoLBench	DeepSWE
Planned tasks	150	99	100	113
No memory	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
FAISS Flat	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeL	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeRepair	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ReasoningBank	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ACE	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ContextGraph	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Graph – Flat (pp, CI)	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 21: Combined success on independently specified held-out checks. Cells report successes/eligible and assessed/eligible. Report sampled and eligible counts for each benchmark; the final row gives the paired difference and 95% interval. Table 33 separates the outcomes.

Method	Verified	Related-Lite	LoLBench	DeepSWE
No memory	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
FAISS Flat	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeL	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeRepair	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ReasoningBank	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ACE	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ContextGraph	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
CG – Flat (pp, CI)	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

G.2 SEPARATE CHECKS, JOINT CHECKS, AND RELATION INFORMATION

The development study follows the three design questions in Sections 4.3–4.5, using 24 tasks and three repetitions. Content interventions compare how text and executable examples retain a trigger (Table 26). H runs the current and historical checks separately; I replaces the historical check with a joint check. Both receive the same history and current program, with two check slots, the same starting state, and matched execution and feedback budgets. C–B tests supplying relations instead of

reconstructing them, with the composer held fixed. C–G changes the composer while retaining the bindings; F–C adds supplied execution observations to test how feedback changes the repair. These contrasts examine triggers, composition, and feedback on the development panel.

Table 22: Development ablations on 24 targets with three repetitions. B/C share a composer; C/G share bindings. H/I compare separate and joint checks with the same two slots and feedback schedule. Report mean $\pm$ SD and paired intervals. Check readiness counts valid delivered checks; Appendix I.1 specifies the comparisons.

Variant	Check ready	Joint	Combined	Regressions
A. Flat records; no composition	n/a	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
B. Reconstruct all bindings; shared composer	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
C. Supplied bindings; same composer	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
D. C with input bindings reconstructed	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
E. C with operation bindings reconstructed	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
F. C + supplied execution observations	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
G. C’s bindings; free-form composer	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
H. Current + separate historical check	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
I. Current + joint check; same feedback	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

G.3 WHERE THE METHOD HELPS AND WHAT IT COSTS

Before running the methods, group tasks by whether the adapters support their operations: supported, outside scope, or uncertain. Compare methods on the same tasks within each group, including failed preparation and text fallback. This distinguishes a lack of coverage from a failed repair. Cost includes source construction, program preparation, retrieval, applicability assessment, check construction and execution, repair attempts, and verification. The retry and amortization tables report how these costs change as the bank is reused.

Table 23: Coverage and cost by benchmark. Preparation, adapter support, and joint delivery use all planned targets. Combined differences use the independent behavioral sample, grouped by support before any run. USD per solution includes preparation and all repair attempts.

Benchmark	P_q ready	Supported	Joint delivery	CG–Flat Combined (pp)	CG/Flat USD/solve
Verified150	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Related-Lite99	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
LoLBench100	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
DeepSWE113	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

H EXPERIMENT 1 PLAN: MATCHED REPAIR EFFECTIVENESS

These planned experiments use a common agent and DeepSeek V4 Pro backbone, with Qwen3-Embedding-8B for new embeddings. Red TBD cells mark results still to collect; black coverage counts are existing measurements. Each patch is graded by its benchmark’s verifier. Paired intervals resample targets while keeping repeated attempts together. Report assessed and planned counts; infrastructure failures remain unassessed. Cost includes preparation, selection, checks, repair, and verification.

Memory settings. The primary comparison fixes each method’s memory before target evaluation. All methods use the same historical sources, with their own memory construction adapted to the common agent. The same backbone performs extraction, selection, composition, and repair. The sequential-update comparison starts from these banks, then permits updates after submission under the same feedback and at least three fixed task orders. Table 20 reports the four-benchmark comparison; Appendix F.2 describes the shared program-preparation step.

Table 24: Questions addressed by the additional experiments. Green additions: Appendix L.

Stage	Question	Experiment and evidence	Primary outcome
1	Which details must memory preserve?	Content and source-utility interventions in Tables 26–27.	Joint; Combined
2	What does condition composition add?	Separate versus joint checks (H/I), supplied relations (B/C), and composer choice (C/G) in Table 22.	Joint; Combined
3	How does execution feedback help?	Source observations in Table 26; supplied check observations (C/F) in Table 22.	Combined; regressions
4	Does the method improve repair?	Four-benchmark resolution (E07), independent behavioral completeness (E15), and applicability (E16).	Official; Combined; omissions
5	When is experience reuse effective?	Retrieval and source coverage, second-backbone results, retries, and amortized cost (E08/E09).	Utility@3; USD/solution; time

I EXPERIMENT 2 PLAN: CONTENT AND MECHANISM ABLATIONS

The development panel uses 24 targets from the 70-task development partition, covering at least four repositories and four operation families. Task/source pairs are selected from public evidence before the new repairs are observed. Each arm has three fresh repetitions (72 attempts), with common source IDs, matched budgets, and balanced arm order. The evaluation suite is fixed in advance and includes joint requirements and originally passing neighbors. A regression means failing at least one of these neighboring checks. All arms share program preparation; preparation and delivery failures stay in the assigned panel. Experiments using researcher-prepared programs are reported separately from automatic preparation.

The seven-method panel first measures Official, Joint, Preserve, and Combined on this suite. The content, relation, composition, and retrieval studies then vary one part of the memory interface at a time.

Table 25: Seven-method behavioral comparison. Entries report mean task success and sample SD across three repetitions. Paired intervals resample targets. Repeat the panel on the second backbone.

Method	Official	Joint	Preserve	Combined
No memory	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
FAISS Flat	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeL	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeRepair	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ReasoningBank	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ACE	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ContextGraph	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 26: Which representation carries the missing requirement? Each task uses the same source, tools, and budget. Inputs are the requirement r , historical implementation H , witness W , and source observations E^- , E^+ . All repairs receive the same private joint and preservation checks. The last two rows isolate supplying source observations.

Delivered memory	Official $\uparrow$	Joint $\uparrow$	Preserve $\uparrow$	Combined $\uparrow$
None	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Requirement r	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Historical implementation H	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
$r + H$	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
$r + W$	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
$r + W + E^- + E^+$	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 27: Which sources help? On 12 development targets, select four distinct sources using dense, lexical, operation-connected, and random same-repository retrieval. Test each source alone for three repetitions (144 attempts), against matched no-memory controls. Group sources by their relation to the current requirement; report pair counts, changes in Combined, and regressions.

Source/current relation	Source pairs	Δ Combined	Δ regressions
Adds an omitted requirement	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Preserves already-satisfied behavior	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Conflicts with the current requirement	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
No established operation connection	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

I.1 GRAPH RELATIONS AND RETRIEVAL

Relations and composition. Table 22 uses the same 24 tasks, three repetitions, source records, applicability decisions, check count, validator, and total budget. B reconstructs all bindings from text; C receives the stored bindings. D/E reconstruct only one withheld binding family. B–F share the same adapter. G receives C’s bindings but constructs a check with free-form generation. F additionally receives execution observations before editing. All arms retain ordinary execution tools. Explanations and tool outputs respect the same withheld-binding masks as the memory input.

Separate and joint checks. H runs the current program plus a standalone historical check; I runs the same current program plus a joint check. Both receive identical history, bindings, environment, starting state, and remaining budget. Each has two check slots, matched execution limits, feedback rounds, and observation format. Both historical and joint checks use requirements applicable to the current version. The independent evaluator tests both kinds of behavior on every repair. Report Combined and the rate of repairs for which separate checks pass but the joint check fails, using the assigned targets and their assessment counts. This directly tests the value of combining conditions.

Table 28: Can retrieval find useful experience? Panel A uses the four-source pools; Panel B uses the full bank on 24 targets. Rankings are fixed before measuring standalone utility on the union of the selected sources. Utility@3 sums standalone changes in Combined and divides by three; empty slots contribute zero. Composable@3 counts targets with at least one executable joint condition. Packet Combined uses three fresh repetitions with the same composer and budget; selection cost includes every retrieval call.

Candidate ranking	Utility@3	Composable@3	Combined	Retrieval ms
A. Controlled four-source pools; 12 development targets				
FAISS dense similarity	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
BM25	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense + BM25 reciprocal-rank fusion	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense + soft operation reranking	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense with a required operation match	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
B. Full source bank; 24 development targets				
FAISS dense similarity	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
BM25	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense + BM25 reciprocal-rank fusion	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense + soft operation reranking	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense with a required operation match	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

J EXPERIMENT 3 PLAN: GENERALIZATION, ROBUSTNESS, AND COST

Repeat the seven-method behavioral panel on a second backbone, fixing its budget before execution and reporting each model separately. For sensitivity, vary retrieval count $k \in \{1, 3, 5\}$ and record-token allowance $B \in \{B_0/2, B_0, 2B_0\}$ one at a time for ContextGraph and Flat. Keep the total selection-plus-repair budget fixed. Report delivered sources, tokens, joint delivery, Combined success, and cost. A complete record that exceeds the allowance is rejected rather than truncated.

Table 29: Retry efficiency, reported separately for each benchmark. Retry each method's unresolved tasks for three passes with memory fixed and the same feedback policy. Report cumulative resolutions, total tokens and USD per solution, and active time to resolve 10% of the task list (rounded up). Include all attempts, preparation, checks, verification, and one-time construction. An unreachable threshold or cost per solution with no solutions is reported as n/a.

Method	Pass@1	Pass@2	Pass@3	Tokens/solve	USD/solve	Time to 10%
No memory	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
FAISS Flat	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeL	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeRepair	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ReasoningBank	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ACE	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ContextGraph	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 30: Experience coverage across benchmarks. The existing-bank column counts targets with at least one witnessed source from the same repository. Expand the bank from training data and pre-base history and develop adapters before evaluation. New witnesses require a source fail-to-pass contrast. Expanded coverage and joint delivery use all planned targets; delivery also requires a working observer. Count text fallback, rejected requirements, and uncertain cases separately.

Benchmark	Existing bank	New witnesses	Expanded coverage	Joint delivery
Verified150	133/150	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Related-Lite99	91/99	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
LoLBench100	2/100	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
DeepSWE113	0/113	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 31: Useful coverage versus ranking competition. With $k = 3$, add 0, 4, or 16 same-repository distractors to each four-source pool. Then replace the source with the highest measured standalone utility, retaining 20 sources. Fix the pools before three fresh repetitions per cell. Report retention for Flat/Graph (F/G) and Combined. Compare source removal with the 20-source distractor condition.

Pool condition	Size	Retained F/G	Flat Combined	Graph Combined
Original candidate pool	4	<i>TBD/TBD</i>	<i>TBD</i>	<i>TBD</i>
Add 4 distractors	8	<i>TBD/TBD</i>	<i>TBD</i>	<i>TBD</i>
Add 16 distractors	20	<i>TBD/TBD</i>	<i>TBD</i>	<i>TBD</i>
Replace best source, fixed size	20	n/a	<i>TBD</i>	<i>TBD</i>

Table 32: Construction cost as memory is reused. For each benchmark and method, report $[C_{\text{build}} + C_{\text{solve}}(N)]/R(N)$ at the listed first-pass prefixes. Build cost includes source construction, execution, embedding, and indexing; solve cost includes preparation and unsuccessful attempts. $R(N)$ counts official resolutions. The crossover is the first measured prefix with at least as many solutions as control and lower cost per solution. Unavailable prefixes and zero-solution cells are n/a.

Method	Build USD	USD/solve at 25	At 50	At 100	First crossover
No memory	n/a	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	Reference
FAISS Flat	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeL	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeRepair	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ReasoningBank	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ACE	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ContextGraph	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

K INDEPENDENT CONFIRMATION: COMPLETENESS AND APPLICABILITY

The development panels select the method; the following tests establish its scope. Their manifests, annotation rules, and endpoints are sealed before new solver outcomes are visible. They do not reuse the 24 development tasks.

K.1 INDEPENDENT HELD-OUT BEHAVIORAL COMPLETENESS

After fixing the method, sample up to 40 untouched tasks per benchmark, using a fixed seed and public repository/operation groups. Exclude tasks previously used to develop prompts, adapters, or the method. Report the available sample and each benchmark’s number of behaviorally eligible tasks.

Two independent annotators specify requirements from the current issue, version-matched documentation and tests, and pre-base history. They identify current behavior, applicable historical requirements, combined triggers, and originally passing neighbors. A third reviewer resolves disagreements; uncertain requirements remain unassessed. The suite is sealed before new patches are seen and kept private from the solver. Targets and expected outcomes are selected independently of ContextGraph’s retrieved or generated checks. Programs observe meaningful behavior, such as selected rows or a converted time.

A task is behaviorally eligible when at least one additional historical requirement can be assessed. Combined requires official success and all applicable joint and preservation checks on the same patch. Omission burden counts official successes that fail an applicable requirement, divided by eligible tasks. Report Official, Joint, Preserve, Combined, and omission burden together, with sampled, eligible, assessed, and paired counts. Results are separated by benchmark and repository group. First-pass patches from the main comparison can be reused when the private suite was sealed before those outputs and the experiment settings match.

Table 33: Independent held-out behavioral evaluation. Repeat for each benchmark with up to 40 untouched tasks. Cells give count/eligible and rate; state sampled, eligible, assessed, and jointly assessed counts. Omission counts official successes failing an applicable requirement, divided by eligible tasks. Table 21 summarizes Combined.

Method	Official	Joint	Preserve	Combined	Omission
No memory	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
FAISS Flat	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeL	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ExpeRepair	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ReasoningBank	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ACE	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ContextGraph	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

K.2 APPLICABILITY AND HARMFUL TRANSFER

Evaluate applicability decisions on all retrieved requirement/target pairs in the held-out sample, including accepted, rejected, and uncertain candidates. Independent annotators judge current validity without seeing the assessor’s decision or repair outcome.

A separate challenge set contains 60 applicable, 60 contradicted, and 30 ambiguous pairs. It covers changed APIs, incompatible types or roles, superseded behavior, and insufficient specification. Report natural retrieval and challenge-set results separately. False acceptance is the fraction of contradicted requirements accepted; useful acceptance is the fraction of applicable requirements accepted. Also report uncertain requirements enforced and the fraction of accepted requirements that are wrong. Intervals preserve source and target clusters.

For repair outcomes, compare applicability checking with bypassing that check on the same eligible tasks. Hold source IDs, records, composer, check budget, and total budget fixed. The private behavioral suite measures preservation, Combined success, and omissions for both arms. Record assessment cost and memory delivery alongside these outcomes.

Table 34: Does checking applicability retain useful history and reject obsolete requirements? Panel A reports decisions on naturally retrieved and challenge-set pairs. Panel B compares the resulting repairs on the private behavioral suite, with assessment cost and memory delivery reported alongside the outcomes. **Green additions: Appendix L.**

Candidate population	Pairs	False accept	Useful accept	Uncertain enforced
Natural retrieved pairs	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Separate challenge strata	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Challenge: applicable	<i>TBD</i>	n/a	<i>TBD</i>	n/a
Challenge: contradicted	<i>TBD</i>	<i>TBD</i>	n/a	n/a
Challenge: ambiguous	<i>TBD</i>	n/a	n/a	<i>TBD</i>
Repair policy	Official	Preserve	Combined	Omission
ContextGraph with applicability check	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Without applicability check	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
No memory	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Text memory + applicability check	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

L DEFINITIONS OF THE GREEN TABLE ADDITIONS

Green entries add conditions or measurements to tables with fewer than four data rows or four columns. Headers do not count as data rows; the method or metric label counts as a column. Each comparison panel meets the same minimum. Green TBD cells are results to collect; the original entries remain unchanged.

Historical requirement on a fourth task (Table 2). Add a Django12663 pair starting from the original target: no memory versus ContextGraph using source13590. Both agents receive the same public current program, tools, backbone, and repair budget; only the memory arm receives the historical requirement and its checks. Run one complete repair per arm, as in the existing case rows. A cell passes when the patch satisfies both the current lazy-User behavior and the named-tuple range requirement on the withheld nonempty joint queries. Report official verification alongside it. These cells require new paired repairs, not the earlier refinement outcomes.

Separate-history control (Table 4). Add the same history and current-program feedback as in the recorded pair, plus standalone named-tuple and ordinary-tuple historical checks. Keep their historical input objects; do not insert the current lazy User into them. Match the composed arm’s supplied check count, feedback rounds, execution instructions, and total repair budget. Evaluate the submitted patch on the same official, historical, joint, ordinary-subclass, and mechanical checks. Each cell reports the corresponding passed/total checks, or the official verdict. This control asks whether the interaction adds value beyond separate historical checks; recover the recorded settings before running it.

Fourth retry pass (Table 6). From each arm’s third-pass unresolved set, allow one further attempt under the same model, per-attempt limits, tools, and fixed episodic bank. Keep a separate continuation record and the earlier predictions. Report cumulative official resolutions out of the original 99 tasks, paired memory-only and control-only wins, and the two-sided exact McNemar test. No newly solved target is added to memory during this comparison.

Pool coverage and cost (Table 7). For each frozen 50/100/200/300-summary pool, source coverage is the percentage of the 98 planned targets with at least one admitted same-repository summary. Compute it before repair. USD/resolution divides memory construction, retrieval, and all first-pass repair and verification costs by official resolutions. Include unsuccessful and interrupted attempts’ incurred costs; use the original pool-specific logs rather than borrowing another run’s costs.

Retry accounting (Table 10). Attempts/task is the total number of solver attempts that started, including failed and budget-interrupted retries, divided by the number of planned tasks in that arm’s frozen manifest. Recover it from the original logs. Report the total and denominator with the mean; an environment failure before the solver starts does not count as an attempt. Missing logs remain unassessed.

Random-strategy control (Table 12). Sample from the same 3,183-strategy pool using a task-specific seed derived from seed zero and the target ID. Match the archived FAISS arm’s number of delivered strategies, formatting, model, tools, and \$10 per-task cap on the same 50 targets. Report completed/planned, official resolutions, the resolved/completed rate, and total recorded cost. This tests selection against random access to the same experience pool.

Graph-vector coverage (Table 13). For each node label, count nodes with a stored vector of exactly 3,072 entries in the archived graph snapshot. The total-node cell counts distinct such nodes; the edge row is n/a because this column measures node embeddings. Query the historical snapshot read-only; these counts do not describe the new Qwen witness graph.

Cost in the new backbone comparisons (Tables 16, 17, and 18). Tokens/task is total input plus output tokens for memory construction, selection, and repair, divided by planned targets; cached input tokens count once. USD/resolution includes the same model work and billed execution and verification, divided by official resolutions. Include unsuccessful attempts and amortize one bank

build over the evaluated target list. Use the reporting budget of the SWE-Context table and first-pass runs for Gemini and Codex. For Gemini’s token row, relative gain is the reduction $(T_{\text{none}} - T_{\text{CG}})/T_{\text{none}}$. Recover the benchmark, manifest, and accounting scope before populating these cells; the separate 236K/201K report is not assigned to Gemini without that match.

FAISS Flat for Codex (Table 18). Use the same Codex/GPT-5.5 agent, confirmed target manifest, source bank, and total budget as the two existing arms. Deliver the same three source IDs as ContextGraph as flat records, without graph relations or composed checks. Report first-pass and three-attempt official resolutions with explicit task denominators, plus first-pass token and cost measurements. Resolve the existing 20-versus-30 sampling units from raw records before a matched run; do not compute a paired gain from the two supplied fractions alone.

Outcome mapping (Table 24). The new column identifies the reported outcome for each research question. Joint, Combined, regressions, and omissions use the common behavioral suite; Utility@3 measures selected-source utility. USD/solution and time to the resolution threshold use the efficiency definitions already specified. This is a definition column, not another experiment.

Applicability strata and controls (Table 34). Separate the existing challenge set into its 60 applicable, 60 contradicted, and 30 ambiguous planned pairs. Report the assessed count and useful acceptance, false acceptance, or uncertain enforcement in the corresponding stratum; rates whose reference class is absent are n/a. The aggregate row continues to summarize the same set. For repair, add no memory and text memory with the applicability check. The latter supplies accepted requirements as text, with the same source IDs, preparation, tools, and total budget, but no supplied composed checks. Evaluate all four policies on the same private held-out suite and report Official, Preserve, Combined, and Omission. Reuse an existing baseline only when task IDs, repetition, starting state, and budget match this comparison.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing, including drafting and revising the abstract.